METODOLOGIA

Influência da especificação de requisitos na segurança e na correção funcional de código gerado por modelos de linguagem

Andre Lucas Francino Castelo Branco

Orientador: Mauro José Araújo de Melo

Engenharia de Software - iCEV

Versão parcial para orientação - 3 de novembro de 2026

3 METODOLOGIA

3.1 Delineamento e questões de pesquisa

O estudo compara código Python/FastAPI gerado por modelos de linguagem sob quatro condições de especificação. A avaliação é quantitativa e examina separadamente o atendimento funcional e propriedades de segurança. A inspeção das falhas de execução complementa as medidas, permitindo distinguir defeitos da implementação, incompatibilidades de bibliotecas e insuficiência da infraestrutura de teste.

A investigação é orientada por três questões. QP1: como o detalhamento funcional se relaciona com as falhas de segurança observadas? QP2: o acréscimo de orientações genéricas ou de requisitos específicos de segurança altera a frequência e os tipos de falhas? QP3: como a aprovação funcional varia entre essas condições? A terceira questão permite examinar uma possível perda funcional associada à especificação de segurança, sem pressupor sua ocorrência.

O piloto já executado reúne 72 gerações, distribuídas entre três tarefas, quatro níveis de especificação, dois modelos e três repetições. Cada geração é uma unidade de análise. Os casos de teste integram a avaliação dessa unidade; sua quantidade não aumenta o número de gerações disponíveis. As mesmas tarefas são utilizadas nos quatro níveis, e as comparações são feitas dentro de cada combinação de tarefa e modelo.

O desenho compara quatro condições de especificação. Não corresponde ao cruzamento completo de dois fatores independentes de detalhamento funcional e segurança. A etapa principal permanece planejada e é tratada separadamente na subseção 3.7.

3.2 Variáveis e condições de comparação

A variável manipulada é o nível da especificação. N0 contém uma solicitação funcional mínima. N1 detalha a função, sem orientação explícita de segurança. N2 mantém N1 e acrescenta uma diretriz genérica de boas práticas e OWASP. N3 mantém N1 e acrescenta requisitos específicos de segurança. Todos os prompts solicitam resposta somente com código Python.

Quadro 2 - Relação entre questões e comparações

Questão	Comparações	Medidas observadas
QP1	N0 e N1, por tarefa e modelo	Achados estáticos e falhas comportamentais por propriedade
QP2	N1 e N2; N1 e N3; N2 e N3	Frequência de falhas observadas e categorias CWE
QP3	N1, N2 e N3 no contrato detalhado; N0 a N3 nas capacidades mínimas	Aprovação funcional, reprovação e avaliação inconclusiva

Fonte: elaborado pelo autor (2026).

As variáveis de resposta são a presença e a contagem de achados estáticos, as categorias CWE atribuídas, o resultado dos testes comportamentais e a aprovação funcional por grupo de testes. Tarefa e modelo são fatores de agrupamento. Tempo de coleta, versões de bibliotecas e configuração dos verificadores são registrados como condições de execução.

A comparação funcional usa dois conjuntos de critérios. O primeiro verifica capacidades operacionais mínimas com as adaptações de interface documentadas. O segundo verifica o contrato detalhado comum a N1, N2 e N3. O resultado desse segundo conjunto também pode ser observado em N0 como referência de capacidade, mas não representa descumprimento de uma exigência ausente de seu prompt.

3.3 Amostra e procedimento de geração do piloto

O piloto abrangeu login por e-mail e senha (T01), busca de produtos com filtros (T02) e consulta de pedidos por identificador (T06). As tarefas foram escolhidas para examinar proteção de credenciais, construção de consultas e autorização. Os identificadores preservam sua posição no catálogo planejado de dez tarefas.

Quadro 3 - Tarefas e instrumentos utilizados

Tarefa	Fraquezas de interesse	Avaliação realizada
T01 - login	CWE-916 e CWE-307	Análise estática, aceitação funcional e bloqueio de tentativas por conta e IP
T02 - produtos	CWE-89	Análise estática e aceitação dos filtros, da ordenação e das respostas
T06 - pedidos	CWE-639	Análise estática, aceitação funcional e acesso ao pedido de outro usuário

Fonte: elaborado pelo autor (2026).

As gerações foram coletadas em 25 de setembro de 2026, utilizando gpt-4.1-mini-2025-04-14 e gpt-5.4-mini-2026-03-17, com 36 registros válidos por modelo. As chamadas receberam prompts individuais e seguiram uma ordem fixa de tarefa, nível e repetição. Os resultados dos testes não foram devolvidos aos modelos para solicitar reparos.

O banco disponível contém 72 gerações sem erro registrado. A temperatura armazenada é 0,7 em todas elas. Para o segundo modelo, há metadados da temperatura enviada, do limite de 1.500 tokens de conclusão e do motivo de término. Para o primeiro modelo, esses metadados completos não foram preservados. O banco não registra cada tentativa de requisição como um evento separado; portanto, os 72 registros não permitem reconstruir um total de chamadas ou retentativas anteriores à obtenção das respostas válidas.

Prompts, respostas brutas, código extraído e resultados foram armazenados em SQLite. A extração removeu cercas de formatação sem corrigir a lógica dos programas. Os arquivos avaliados foram preservados e identificados por SHA-256. Os scripts, dados exportados, relatórios e exemplos de conferência estão disponíveis no repositório da pesquisa: https://github.com/Andre-PI/tcc-requisitos-seguranca-llms.

3.4 Instrumentos e critérios de avaliação

A análise estática utilizou Bandit 1.9.4 e Semgrep 1.178.0 nas 72 gerações. Para cada achado, foram registrados ferramenta, regra, CWE, severidade, linha e mensagem. O Semgrep utilizou o conjunto remoto p/python. As ocorrências de cada ferramenta são mantidas separadamente, pois regras distintas podem sinalizar o mesmo trecho. A categoria CWE da tarefa expressa uma fraqueza de interesse; sua presença depende da evidência obtida na análise.

Os testes dinâmicos utilizaram diretórios e bancos próprios para cada amostra. As adaptações permitidas incluíram montar roteadores em uma aplicação FastAPI, fornecer módulos externos declarados, inserir dados e substituir dependências de autenticação existentes por identidades controladas. Nenhuma verificação de autorização ou política de bloqueio foi acrescentada ao endpoint. O ambiente principal utilizou Python 3.14.2, FastAPI 0.141.1, Pydantic 2.13.5 e SQLAlchemy 2.1.0; as demais versões constam no arquivo de dependências.

Em T06, cada uma de duas identidades acessou primeiro o próprio pedido, estabelecendo o controle positivo. Em seguida, cada identidade solicitou o pedido da outra. O retorno do mesmo conteúdo obtido pelo proprietário foi considerado exposição. A rejeição por 403 ou 404 atendeu ao critério básico de impedir o acesso cruzado; a exigência de N3 por 404 foi registrada à parte. Erros de execução ou falhas no acesso legítimo tornaram inconclusiva a avaliação da autorização.

Em T01, foram aplicadas 25 tentativas com senha incorreta em uma janela máxima de 60 segundos. O teste por conta variou os IPs e manteve a conta; o teste por IP variou as contas e manteve o endereço. Após cada sequência, uma credencial válida foi utilizada, junto de um controle independente, para verificar o bloqueio. O resultado se restringe a esse limiar de tentativas. O prazo padrão de execução comportamental foi de 150 segundos por

amostra.

A aceitação funcional examinou credenciais válidas e inválidas em T01; combinações de filtros, ordenação e valores esperados em T02; e conteúdo dos pedidos, itens e respostas para identificador inexistente em T06. As expectativas foram calculadas a partir dos dados de teste, sem usar a resposta da aplicação como referência. Casos adicionais de robustez foram mantidos em grupo próprio. O executor adotou prazo padrão de 30 segundos por amostra.

Quadro 4 - Classificação dos resultados

Instrumento	Critério favorável	Falha ou impedimento
Aceitação funcional	Todos os casos do grupo aprovados	Ao menos um caso reprovado; avaliação inconclusiva quando o grupo não pode ser concluído
Autorização em T06	Acesso legítimo válido e rejeição dos acessos cruzados examinados	Exposição do pedido de outro usuário; avaliação inconclusiva quando os controles falham
Bloqueio em T01	Rejeição da credencial válida após a série, com controle independente válido	Credencial aceita após a série; avaliação inconclusiva sem controle válido
Análise estática	Ausência de alerta na configuração empregada	Achado registrado para revisão; erro de ferramenta registrado separadamente

Fonte: elaborado pelo autor a partir dos critérios do piloto (2026).

A ausência de alerta estático não é classificada como aprovação geral de segurança. Os resultados comportamentais também são apresentados por propriedade, preservando a diferença entre uma falha observada e a impossibilidade de avaliação. A política de adaptação, os dados de teste e os diagnósticos de execução estão documentados no repositório.

3.5 Métricas e análise dos dados

Para cada combinação de tarefa, modelo e nível, n representa o número total de gerações; A, as aprovadas no grupo funcional; R, as reprovadas; e I, as inconclusivas. As proporções são calculadas por F = A/n, D = R/n e U = I/n, com A + R + I = n. As tabelas apresentam as contagens juntamente com as proporções, mantendo no denominador as aplicações que não puderam ser avaliadas.

Para cada propriedade comportamental, B é o número de gerações com falha observada e I_s é o número de avaliações inconclusivas. A proporção de falhas observadas é B/n, acompanhada de I_s/n. Uma análise restrita aos casos conclusivos pode ser apresentada como resultado adicional, com B/(n - I_s), desde que n - I_s seja maior que zero. As propriedades de bloqueio por conta e por IP são contabilizadas separadamente.

Na análise estática, n_s corresponde ao número de gerações efetivamente analisadas por uma ferramenta, V ao número com pelo menos um achado e H ao total de achados. São

calculados V/n_s e H/n_s, além da distribuição de ocorrências por CWE. Falhas do analisador são informadas em relação ao total de gerações. Esses indicadores descrevem alertas; a proporção de vulnerabilidades confirmadas depende da revisão das ocorrências.

As medidas funcionais e de segurança são relacionadas pelo identificador da geração. Uma aprovação conjunta exige atendimento ao grupo funcional e às propriedades de segurança avaliadas naquela tarefa; resultados inconclusivos permanecem em categoria própria. A quantidade de requisições ou de casos internos não é utilizada como tamanho amostral da comparação entre níveis.

O piloto recebeu análise descritiva, com contagens e proporções por tarefa, modelo e nível. Não foram realizados testes inferenciais para confirmar as questões de pesquisa. O plano inferencial da coleta principal será registrado antes das novas gerações, considerando os contrastes prioritários, o agrupamento por tarefa e modelo e o tratamento dos inconclusivos. A revisão humana estratificada dos achados estáticos permanece pendente.

3.6 Limitações do piloto

A bateria ampliada de aceitação foi definida após a coleta. Algumas convenções, como ordenação ascendente e inclusão dos limites de preço, não estavam integralmente determinadas nos prompts. As versões das bibliotecas também não foram informadas por completo. Por isso, a avaliação funcional está sujeita a convenções do instrumento e incompatibilidades de ambiente. Os diagnósticos de sensibilidade a versões e as intervenções exploratórias de infraestrutura foram registrados separadamente, sem substituir os resultados principais.

N1 estabelece uma base funcional comum a N2 e N3, mas não equaliza a extensão dos prompts. A ordem fixa das chamadas e a ausência de metadados completos de parte da coleta também limitam o controle e a auditoria retrospectiva. Uma cópia das regras p/python obtida depois da coleta não permitiria afirmar que seu conteúdo é idêntico ao utilizado no piloto.

A cobertura comportamental é parcial. Em T01, o procedimento não avalia integralmente hash, temporização, logs e expiração. Em T02, a aceitação não comprova ausência de injeção SQL. Em T06, identidades controladas permitem verificar autorização, mas não validam o autenticador original. A separação por subprocessos constitui isolamento operacional e precisará ser ampliada antes da execução de tarefas com comandos e comunicação externa.

3.7 Coleta principal planejada

O planejamento mantém dez tarefas, quatro níveis, dois modelos e dez repetições por combinação, totalizando 800 novas gerações. As sete tarefas adicionais abrangem upload e download de arquivos, recuperação de senha, execução de utilitário externo, importação YAML, renderização de perfil e requisições a URLs informadas pelo usuário. O piloto será preservado como etapa exploratória, sem incorporação automática à amostra principal.

Antes da coleta, serão definidos os modelos, os prompts, as versões, os critérios de adaptação, as regras locais dos analisadores e o tratamento de falhas. A ordem das chamadas será sorteada com registro da semente e da sequência. Os verificadores serão avaliados com implementações corretas e falhas conhecidas, começando por uma tarefa adicional antes da ampliação completa. A quantidade de dez repetições é uma escolha de escopo e não constitui, por si só, garantia de poder estatístico.